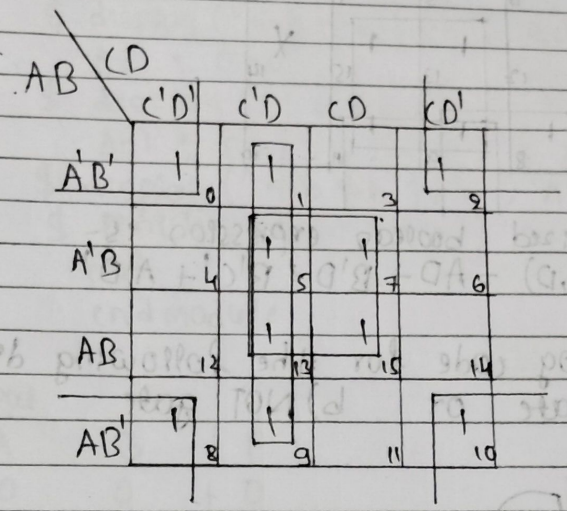
D D M M Y Y Y Y

Assignment - I

01) Minimize the following Boolean function using K-Map
$F(A, B, C, D) = \Sigma(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$

→ Since the given boolean expression has 4 variables, so we draw a 4 variable K-Map.

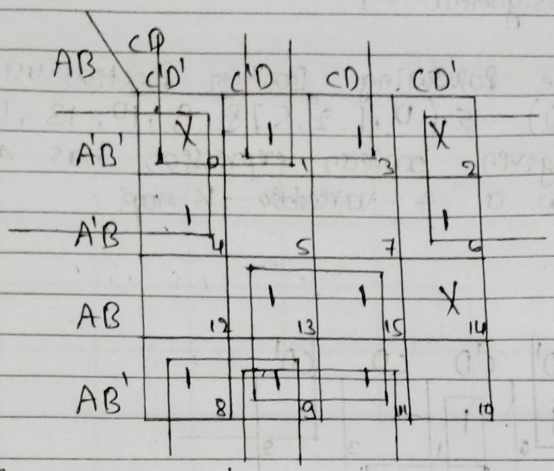


Thus, Minimized boolean expression is -
$F(A, B, C, D) = BD + C'D + B'D'$

02) Minimize the following ~~for~~ boolean function using K-Map using Don't care condition.
$F(A, B, C, D) = \Sigma(1, 3, 4, 6, 8, 9, 11, 13, 15) + \Sigma d(0, 2, 14)$

→ Since the given boolean expression has 4 variables so we draw a 4 variable K-Map.

D D M M Y Y Y Y

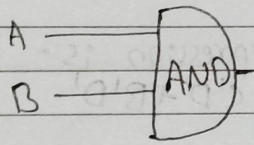


Thus, minimized boolean expression is-

$F(A,B,C,D) = AD + B'D + B'C' + A'D'$

03) write verilog code for the following digital circuits
a) AND Gate or b) NOT gate

→ AND gate



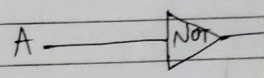

```
Module and_gate (A, B, F);
    input A, B;
    output F;
    assign F = A&B;
    end Module
Module and_gate_tb;
    reg A, B;
    wire F;
```

```
and_gate aa(A
initial begin
$ display("A
$ display("_
  A=0, B=0;
$ display("%
  A=0; B=1;
$ display("%
  A=1; B=0
$ display("%
  A=1; B=1
$ display("%
$ finish;
  end
  end module
```

output

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

NOT gate




```
Module not_g
input A;
output F;
```

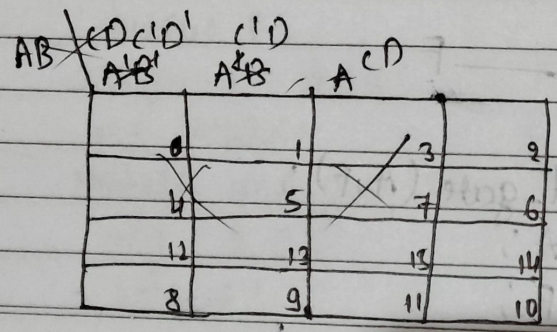
```
assign F = ~A;
end Module
Module not_gate_tb;
    reg A;
    wire F;
not_gate uut (A,F);
    initial begin
$ display ("A | F ");
$ display ("- - - - - ");
    A=0;
$ display ("%b | %b ", A,F);
    A=1;
$ display ("%b | %b ", A,F);
$ finish;
 end
 end Module.
```

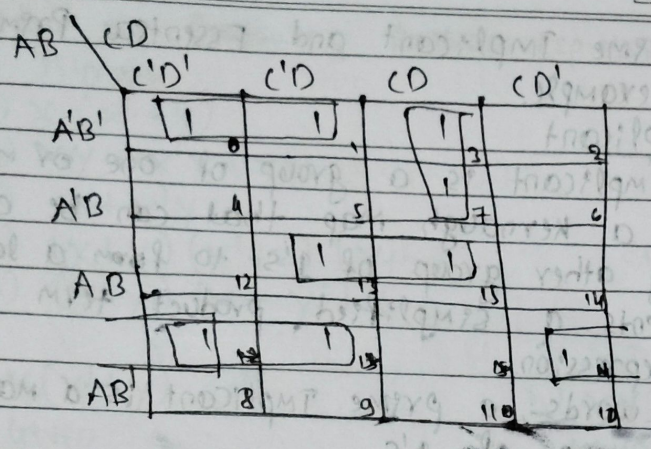
output

A	F
0	1
1	0

04) Simplify the following Boolean expression using K-map

$F(A,B,C,D) = \Sigma(0,1,3,7,8,9,10,13,15)$

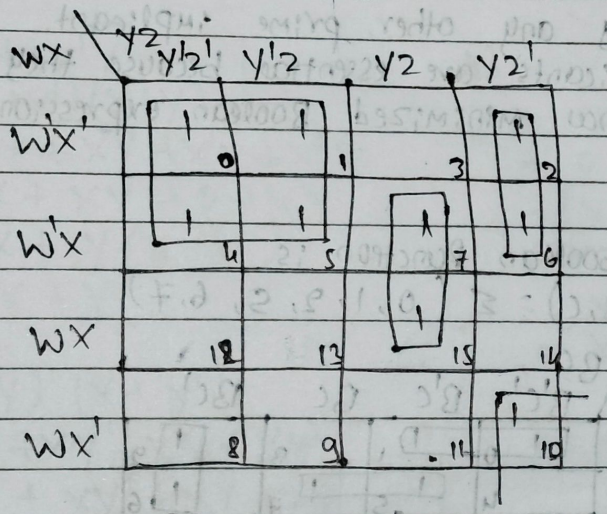




$F = \bar{B}\bar{C} + \bar{A}CD + AB\bar{D} + A\bar{B}D$

OR

$F(W, X, Y, Z) = \Sigma(0, 1, 2, 4, 5, 6, 7, 10, 15)$



$F = \bar{W}\bar{Y} + \bar{W}YZ + XYZ + \bar{X}Y\bar{Z}$

D D M M Y Y Y Y

03) Define Prime Implicant and Essential Prime Implicant. Give an example.

→ **Prime Implicant**

- A Prime Implicant is a group of one or more adjecent 1's in a karnaugh map that can be combined with any other group of 1's to form a larger group.
- It represents a simplified product term in a Boolean expression.
- In other words, a prime Implicant is a maximally combined group of 1's.

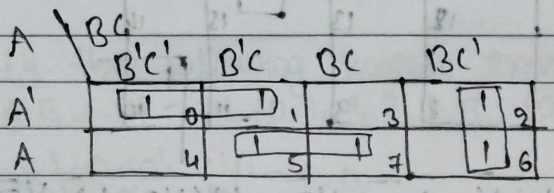
Essential Prime Implicant

- An Essential prime Implicant is a prime Implicant that covers atleast one minterm (1-cell) which is not covered by any other prime implicant.
- Such Implicants are essential because they must be included in the final minimized Boolean expression.

Ex:

Let the boolean function is

$$F(A,B,C) = \Sigma(0,1,2,5,6,7)$$



$$F = \bar{A}\bar{B} + AC + B\bar{C}$$

Implicant.

ve adjecent
bined
er group.
a

Many

cont than
not

be included

a6) Reduce the following Boolean expression to the minimum no. of literals

a) $x(x'+y)$

b) $xy+x'y+yz$

c) $x+x'y$

d) $(x+y)(x'+z)(y+z)$

e) $(x+y)(x+y')$

a) $x(x'+y)$

Given

$= x(x'+y)$

$= xx'+xy$

$= 0+xy \quad [xx'=0]$

$= xy$

b) $xy+x'y+yz$

Given

$= xy+x'y+yz$

$= y(x+x')+yz \quad [x+x'=1]$

$= y+yz \quad [1+z=1]$

$= y[1+z]$

$= y$

c) $x+x'y$

$= (x+x')(x+y)$

$= xx+xy+x'x+x'y$

$= x+xy+0+x'y$

$= x+y(x+x')$

$= x+y$

d) $x+y(x'+z)(y+z)$

$= xx'+xz+yx'+yz(y+z)$

$= 0+xz+yx'+yz(y+z)$

$= (xz+x'y+yz)(y+z)$

$= xzy+x'yy+yzy+xzz'+x'yz+yzz$

$= xyz+xz+x'y+x'yz+yz+yz$

$= y(x+x')+yz+x'y+yz$

$= yz+xz+x'y+yz+yz$

$= yz+xz+x'y$

e) $(x+y)(x+y')$

$= xx+xy'+xy+yy'$

$= x+xy'+xy+yy'$

$= x+x(y'+y)+yy'$

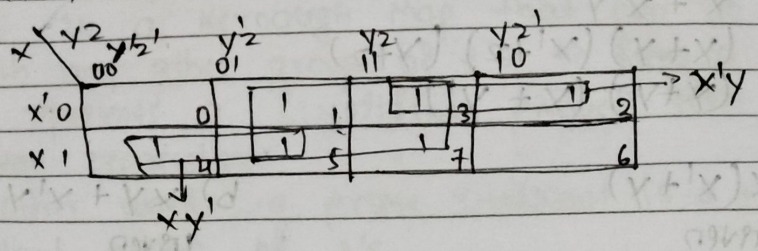
$= x+x+yy'$

$= x+0$

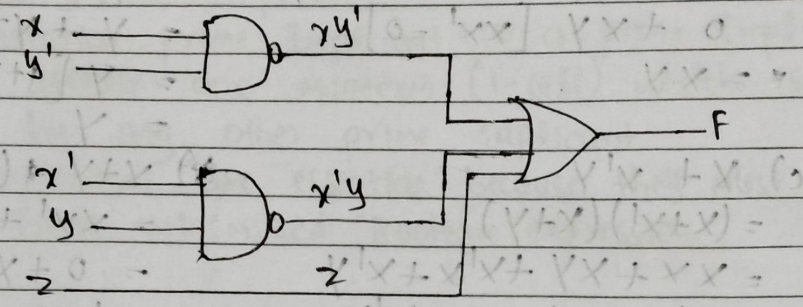
$= x$

07) $F(x,y,z) = \sum(1,2,3,4,5,7)$ construct the K-Map, simplify and convert into NAND gate.

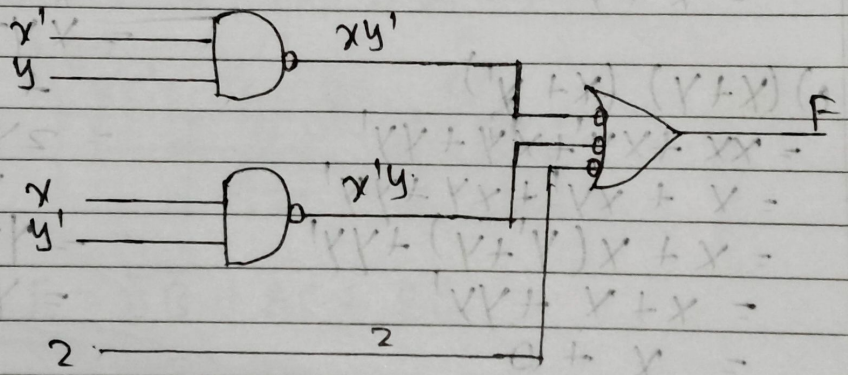
$F(x,y,z) = \sum(1,2,3,4,5,7)$



$F(x,y,z) = xy' + z + x'y$



NAND Implementation



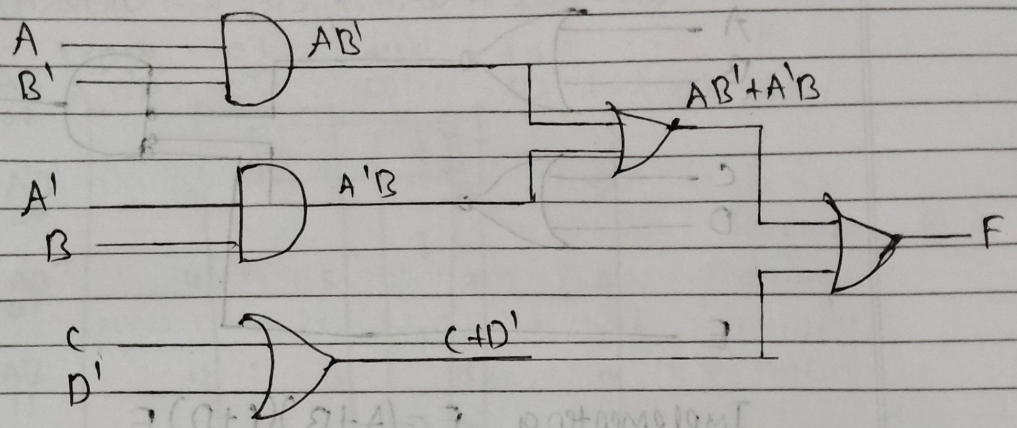
$F = xy' + x'y + z$

08) $F = (A$
NAND
→ $F = ($
A
B
NAN

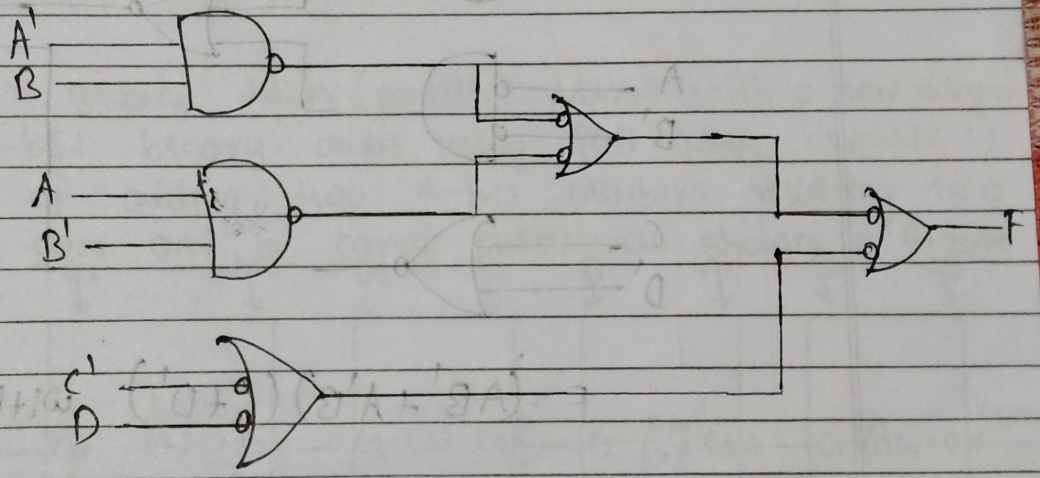
op, simplify

08) $F=(AB'+A'B)+(C+D')$ convert into basic gates then to NAND implementation.

→ $F=(AB'+A'B)+(C+D')$



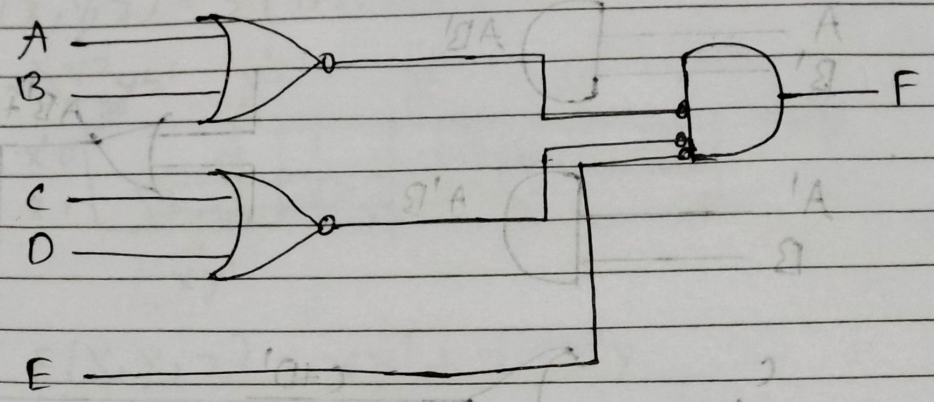
NAND Gate



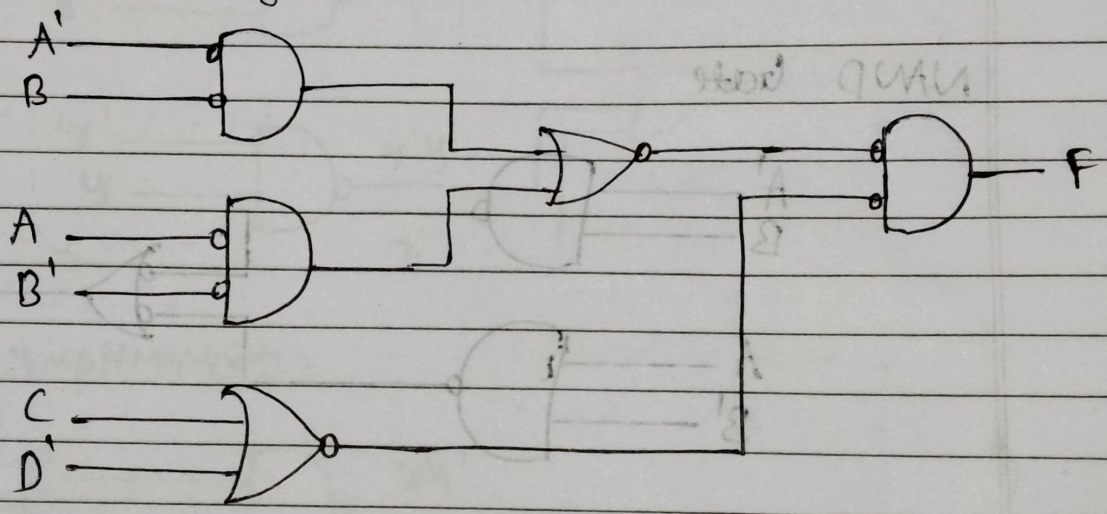
$F = AB' + A'B + C + D'$

09) $F = (AB' + A'B)(C+D')$ convert into basic gates then to NOR Implementation

$F = (AB' + A'B)(C+D')$



Implementing $F = (A+B)(C+D)E$



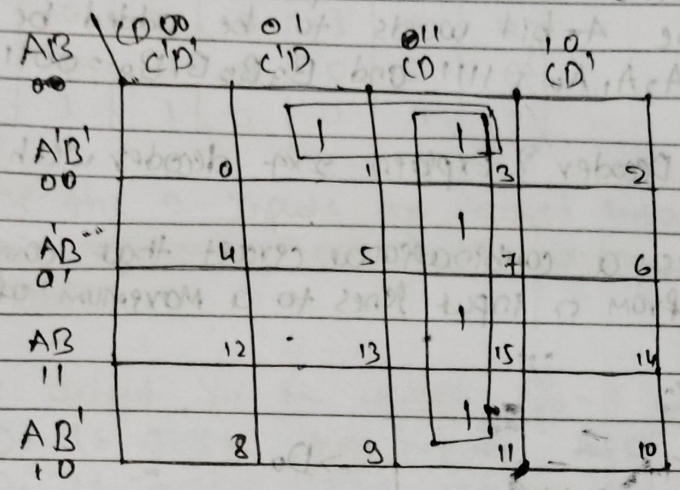
$F = (AB' + A'B)(C+D')$ with NOR gates.

D D M M Y Y Y Y

10) Minimize the following Boolean function using K-map using Don't care condition $F(A,B,C,D) = \Sigma(1,3,7,11,15) + \Sigma d(0,2,5)$

→ Given,

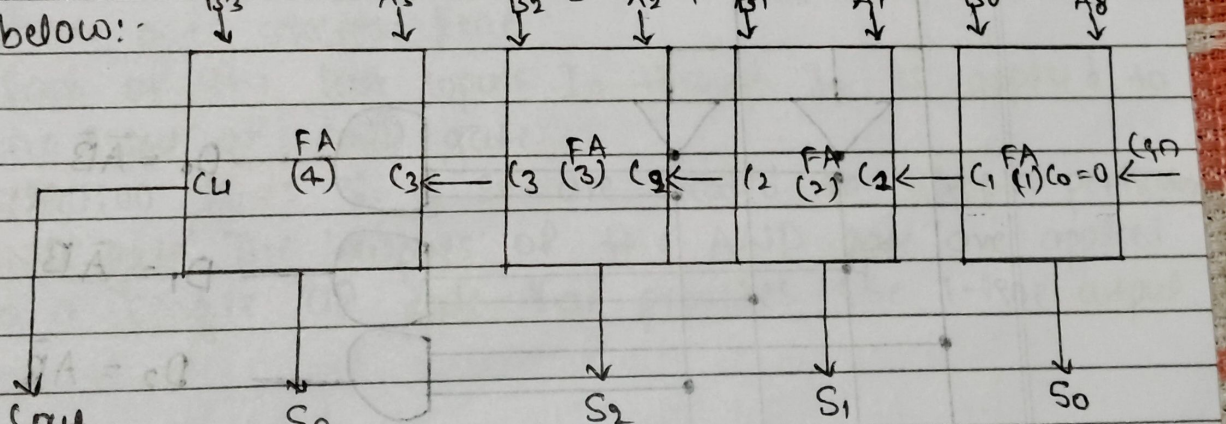
$F(A,B,C,D) = \Sigma(1,3,7,11,15) + \Sigma d(0,2,5)$



$F = \bar{A}B + CD$

11) Explain Binary Adder (parallel Adder) with a neat diagram

→ The 4-bit binary adder using full adder circuits is capable of adding two 4-bit numbers resulting in a 4-bit sum and a carry output as shown in figure below:

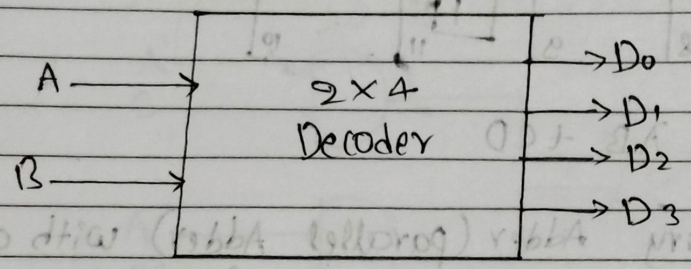


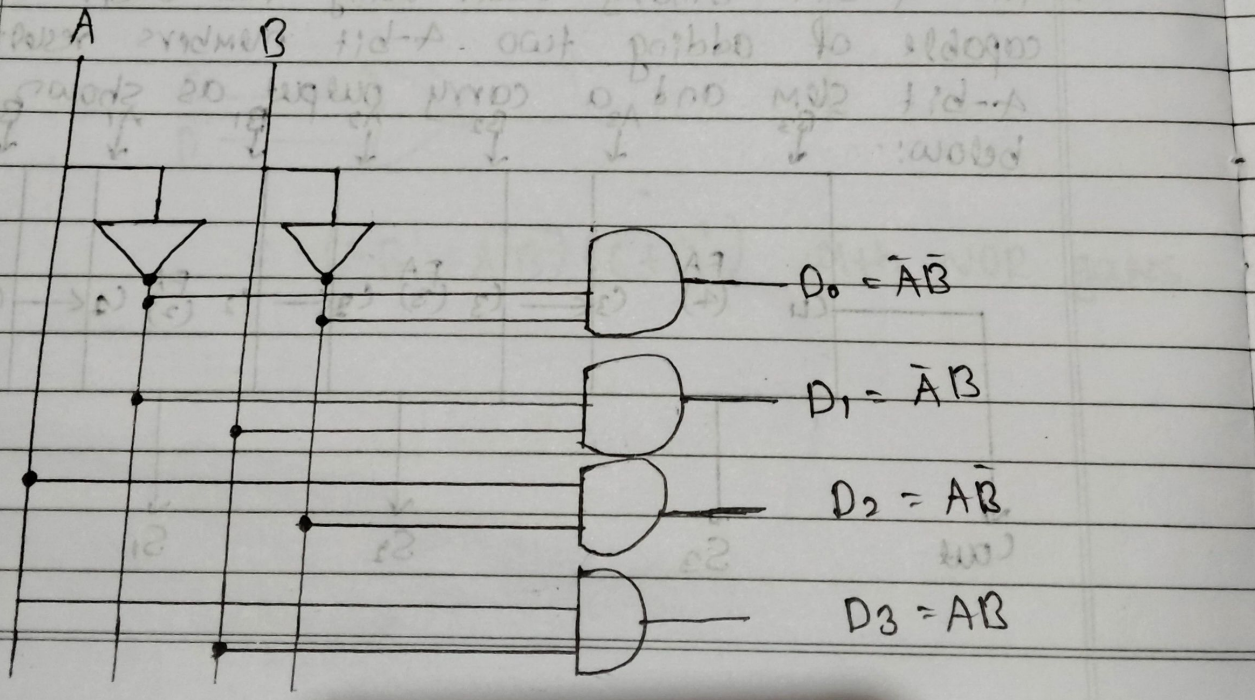
Since all the bits of augend and added are fed into the adder circuits simultaneously and the additions in each position are taking place at the same time, this circuit is known as parallel adder.

Let the 4-bit words to be added be represented by, $A_3A_2A_1A_0 = 1111$ and $B_3B_2B_1B_0 = 0011$

12) What is Decoder? Explain 2x4 decoder with a neat diagram

→ A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2n unique output lines.





Inputs		Outputs			
A	B	d_0	d_1	d_2	d_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

→ Here the 2 Inputs are decoded into 4 outputs, each output representing one of the Minterms of the two input variables.

→ The output Yo is active, $D_0 = 1$ when inputs A=B=0,
D_1 is active when inputs, A=0 and B=1,
D_2 is active when inputs, A=1 and B=0,
D_3 is active when inputs A=B=1.

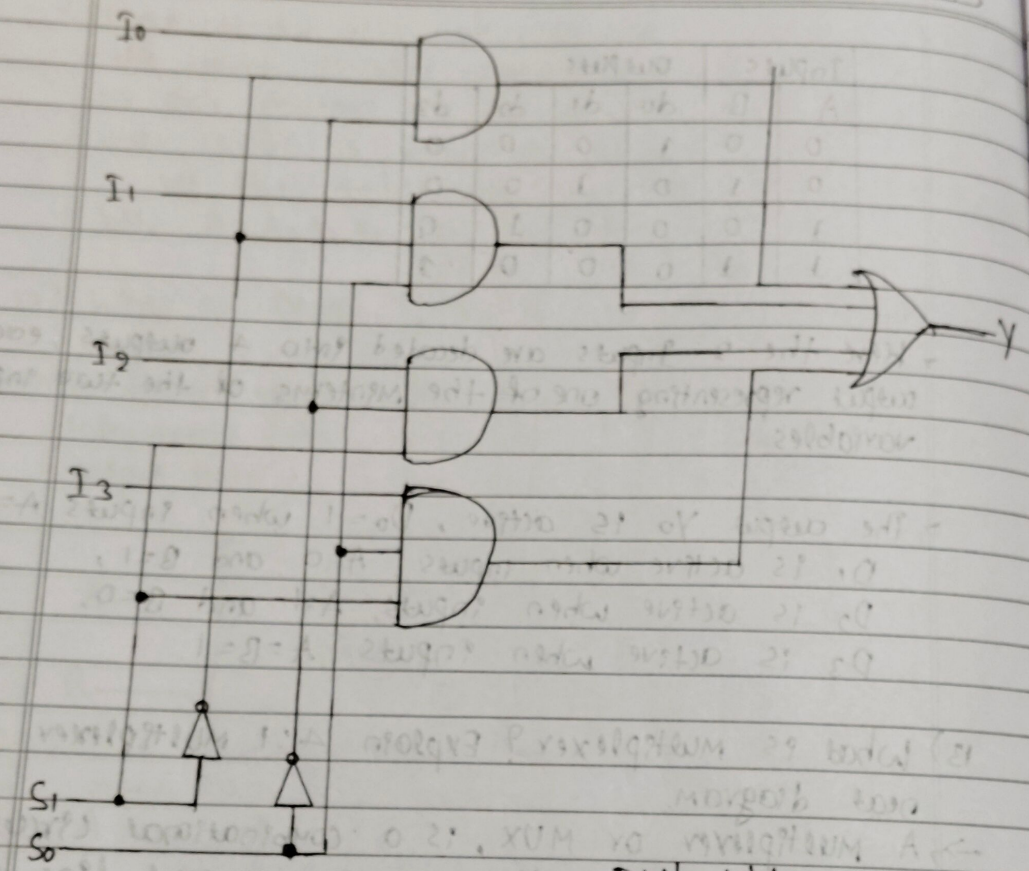
13) What is Multiplexer? Explain 4:1 Multiplexer with a neat diagram.

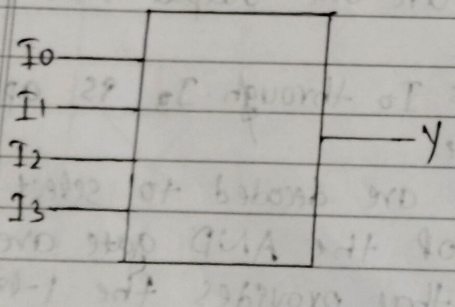
→ A Multiplexer or MUX, is a combinational circuit with more than one input line, one output line and more than one selection line.

→ Each of the four inputs I_0 through I_3 is applied to one input of AND gate.

→ selection lines S_1 & S_0 are decoded to select a perticular AND gate. The outputs of the AND gate are applied to a single OR gate that provides the 1-line output.

D D M M Y Y Y Y





Truth table

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

→ To demonstrate to circuit operation, consider the case when $S_1S_0=10$. The AND gate associated with input I_2 has two of the inputs equal to 1 and the third input connected to I_2.

→ The other
which make
is now equa
the selected
→ The data
$Y = I_0 S_1 S_0$
The data
The data ou
The data outp
when these
data outpu
$Y = I_0$

14) Implement
$f(A, B$
→ variable
select
2^{n-1}
Input

Implemen
Apply
procedur
i) List th
ii) List u
The fo
and th
Any (sr
applying
inputs

→ The other three AND gates have atleast one input equal to 0, which makes their outputs equal to 0. The OR output is now equal to the value of I_2, providing a path from the selected input to the output.

→ The data output is equal to I_0 only if $S_1=0$ and $S_0=0$; $Y=I_0S_1S_0$

The data output is equal to I_1 only if $S_1=0$ and $S_0=1$; $Y=I_1S_1S_0$

The data output is equal to I_2 only if $S_1=1$ & $S_0=0$; $Y=I_2S_1S_0$

The data output is equal to I_3 only if $S_1=1$ & $S_0=1$; $Y=I_3S_1S_0$

When these terms are ORed, the total expression for the data output is,

$$Y=I_0S_1S_0+I_1S_1S_0+I_2S_1S_0+I_3S_1S_0$$

14) Implement the following Boolean function using 4:1 Multiplexer

$$F(A,B,C)=\Sigma(1,3,5,6)$$

→ variables, $n=3$ (A, B, C)

select lines $=n-1=2$ (S_1, S_0)

2^{n-1} to MUX i.e 2^2 to 1 = 4 to 1 MUX

Input lines $=2^{n-1}=2^2=4$ (D_0, D_1, D_2, D_3)

Implementation table

Apply variables A & B to the selected lines. The procedure for Implementation the function are :-

i) List the input of the multiplexer

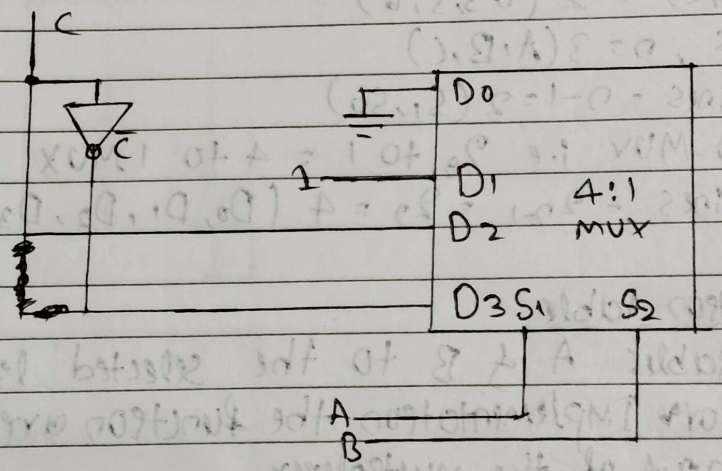
ii) List under them all the minterms in two rows as shown below. The first half of the minterms is associated with A' and the second half with A. The given function is implemented by circling the ~~members~~ minterms of the function and applying the following rules to find the values for the inputs of the multiplexer.

D D M M Y Y Y Y

1) If both the minterms in the column are not circled apply 0 to the corresponding input.
2) If both the minterms in the column are circled, apply 1 to the corresponding input.
3) If the bottom minterm is circled and the top is not circled, apply C to the input.
4) If the top minterm is circled & the bottom is not circled, apply C' to the input.

	D_0	D_1	D_2	D_3
$\bar{C}$	0	(1)	2	(3)
C	4	(5)	(6)	7
	0	1	C	$\bar{C}$

Multiplexer Implementation:



15) Derive characteristics equation for JK and T Flip-Flop and explain in detail.

(A)

D D M M Y Y Y Y

→ ① JK Flip flop

Function Table

J	K	Q_n	Q_{n+1}
0	0	Q	Q
0	1	0	0
0	1	0	0
1	0	1	1
1	1	$\bar{Q}$	$\bar{Q}$

Next-State Table

J	K	Q	Q^+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

K-Map

J \ KQ	$\bar{K}\bar{Q}$	$\bar{K}Q$	KQ	$K\bar{Q}$
$\bar{J}$	0	1	0	0
J	1	1	0	1

$$Q^+ = J\bar{Q} + \bar{K}Q$$

② T Flip-flop

Function Table

T	Q_{n+1}
0	Q
1	$\bar{Q}$

Next-State Table

T	Q	Q^+
0	0	0
0	1	1
1	0	1
1	1	0

K-Map

T \ Q	$\bar{Q}$	Q
$\bar{T}$	0	1
T	1	0

$$Q^+ = \bar{T}Q + T\bar{Q}$$

D D M M Y Y Y Y

16) Derive characteristics equation for D and SR Flip-flop and explain in detail.

i) D - Flip Flop

Function table

D	Q+
0	0
1	1

Next-state table

D	Q	Q+
0	0	0
0	1	0
1	0	1
1	1	1

K-Map

D \ Q	0	0
$\bar{D}$	1	1
D		

$Q^+ = D$

ii) SR Flip-Flop

Function Table

S	R	Q+
0	0	Q
0	1	0
1	0	1
1	1	-

Next-state Table

S	R	Q	Q+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

K-Map

S \ RQ	$R\bar{Q}$	$\bar{R}Q$	RQ	$R\bar{Q}$
$\bar{S}$	0	1	0	0
S	1	1	x	x

$Q^+ = S + \bar{R}Q$

17) Define a BCD-to-excess 3 code converter.

→ Truth table

Inputs (BCD)				outputs Excess-3			
A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

K-Map for W:-

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

$W = A + BD + BC$

K-Map for X:

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	X	X	X	X
10	0	1	X	X

$X = \bar{B}D + \bar{B}C + B\bar{C}\bar{D}$

K-Map for Y:

AB \ CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	X	X	X	X
10	1	0	X	X

$Y = CD + \bar{C}\bar{D}$

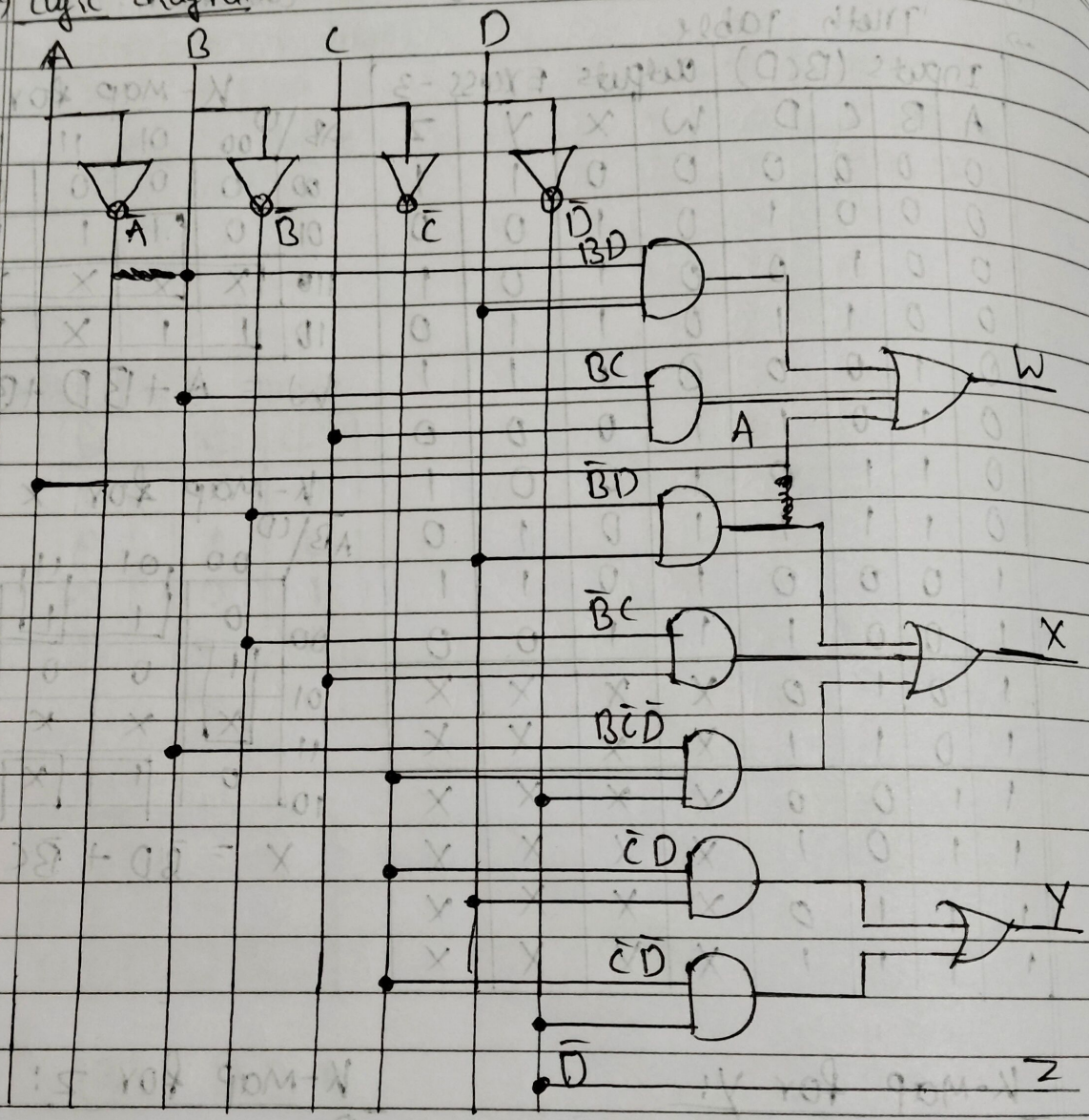
K-Map for Z:

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

$Z = \bar{D}$

D D M M Y Y Y Y

8) Logic diagram



(8) Define Decoder. Describe the working principle of a 3:8 decoder. Draw the logic diagram of the 3:8 decoder with enabled input.

→ A decoder is a combinational circuit that converts binary information from n-input lines to a maximum of 2^n unique output lines.

Working Principle

→ A 3-to-8 decoder is a combinational logic circuit that converts 3 binary input signals into 8 unique output lines. It decodes the binary input into a specific output line that means only one output lines becomes active (logic 1) at a time, while all other outputs remain inactive (logic 0).

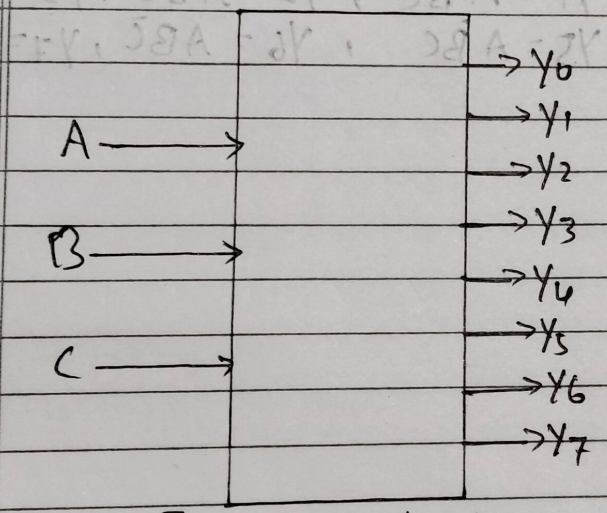


Fig: The 3x8 decoder

The 3x8 decoder, here 3 inputs are decoded into 8 outputs each output represent one of the Minterms of the 3-input variables.

D D M M Y Y Y Y

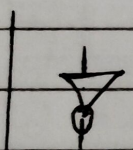
Inputs										
A	B	C	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Fig: Truth table for 3x8 decoder

The output expressions are,

$Y_0 = \bar{A}\bar{B}\bar{C}$, $Y_1 = \bar{A}\bar{B}C$, $Y_2 = \bar{A}B\bar{C}$, $Y_3 = \bar{A}BC$

$Y_4 = A\bar{B}\bar{C}$, $Y_5 = A\bar{B}C$, $Y_6 = AB\bar{C}$, $Y_7 = ABC$



19) Differentiate Latches and Flip-Flops

Latches	Flip-Flops
i) Latches is also a bistable device whose states are also represented as 0 and 1.	i) Flip-Flop is a bistable device it has two stable states that are represented as 0 & 1.
ii) It checks the inputs continously and responds to the changes in inputs immediately.	ii) It checks the inputs but changes the output only at times defined by the clock signal or any other control signal.
iii) It is a level triggered device.	iii) It is edge triggered device.
iv) These are also made of gates.	iv) Gates like NOR, NOT, AND, NAND are building blocks of flip flops.
v) There is no such classification in latches	v) They are classified into asynchronous or synchronous flip-flops.
vi) These can be used for the designing of sequential circuits but are not generally preferred.	vi) It forms the building blocks of many sequential circuits like counters.
vii) Latches doesn't have a clock signal.	vii) Flip-flop always have a clock-signal.
viii) Latches can be build from gates	viii) Flip-flop can be build from latches.
ix) Ex: SR, Latch, D Latch.	ix) Ex: D Flip-Flop, JK Flip-Flop

D D M M Y Y Y Y

20) Implement $Y(A, B, C, D) = \Sigma(0, 1, 6, 7, 8, 9, 10, 11, 12, 14)$ using 16-to-1 multiplexer and 8-to-1 multiplexer.

→

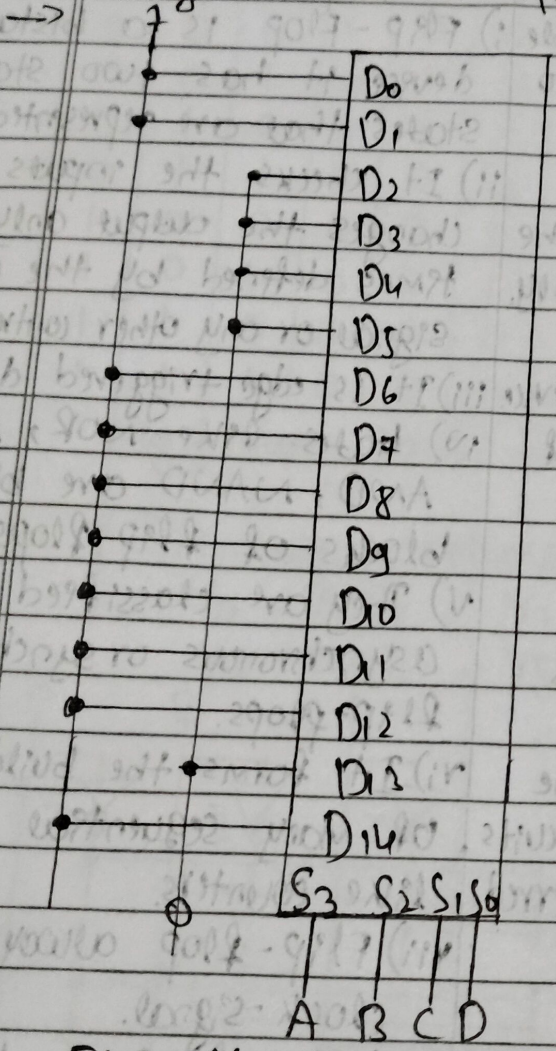


Fig: 16X1 MUX

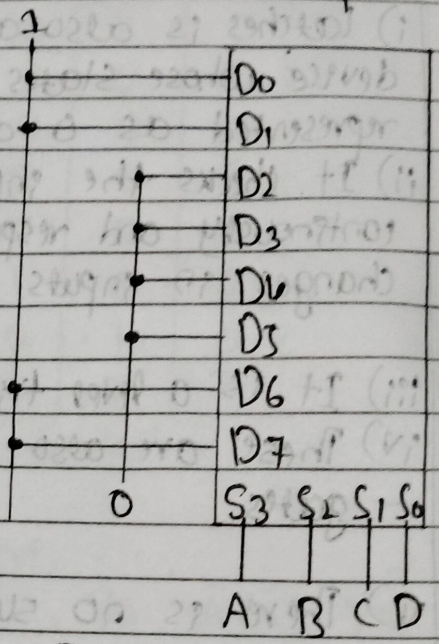


Fig: 8X1 MUX